

III. Background

This section introduces established concepts needed to understand visual action reasoning and logical specifications: vision-language models and Chain of Causation, satisfiability-based verification, controlled natural language, and low-rank adaptation.

A. Vision-Language Models and Chain of Causation

Vision-language models (VLMs) process visual and textual information to generate descriptions or responses about a scene. Vision-language-action models (VLAs) connect such processing to action prediction. Alpamayo-R1 combines vision-language reasoning with trajectory generation for driving. [1]

Visual input, linguistic explanation, and action output serve different roles. The input provides observations; an explanation relates observations to a decision; the output represents the selected action. For example, answering “What is visible?” and “Which action should be selected next?” defines different learning targets. An action output can itself be a linguistic category such as “decelerate,” rather than a trajectory or a steering and acceleration command. Descriptions of action prediction should therefore specify both the available inputs and the representation being learned.

Chain of Causation (CoC) expresses causally connected observations, rationales, and actions relevant to a decision. Alpamayo-R1 aligns these explanations with driving behavior. [1] An illustrative explanation is “the leading vehicle decelerates → the gap may shrink → decelerate to maintain separation.” This example describes an action rationale; it is distinct from statistically identifying causal relationships from observational data.

The first part of this example is an observation, the second an anticipated consequence, and the last an action rationale. These arrows indicate explanatory flow, not the formal implication defined below. Learning from such explanations encourages a model to express these relationships. Whether the stated facts are supported by observations and whether the model selects an appropriate action remain separate questions.

B. Logical Specifications and Satisfiability-Based Verification

1. Variables and Logical Operators

Logical specifications express states and requirements through variables, predicates, and formulas. Propositional variables take truth values; predicates express conditions on objects or quantities. For example, ‘x is nonnegative’ is an arithmetic condition on an integer variable x. Propositional satisfiability (SAT) asks whether a truth assignment satisfies a formula. Satisfiability modulo theories (SMT) also considers background theories such as arithmetic. [2] Z3 supports theories including arithmetic, bit-vectors, arrays, and uninterpreted functions. [3]

The notation below distinguishes propositions from formulas that may combine multiple conditions. In particular, Φ and P need not be single Boolean variables.

Symbol	Meaning	Interpretation
p, q	Propositions	Whether a condition holds
$\neg p$	Negation	p is false
$p \wedge q$	Conjunction	Both p and q are true
$p \rightarrow q$	Implication	If p , then q
Φ	Specification and assumptions	Conditions to satisfy together
P	Property formula	The claim to establish
$\Phi \models P$	Semantic entailment	Every model of Φ also satisfies P

For example, $p \rightarrow q$ is false only when p is true and q is false. When p is false, the implication alone does not determine q . Replacing “if p , then q ” with “if q , then p ” or “if not p , then not q ” therefore generally changes a specification.

2. Satisfiability and Counterexamples

Let Φ combine a specification and its assumptions, and let P denote a property. Satisfiability of Φ establishes the existence of a model satisfying the combined conditions. Property verification can be expressed by checking the counterexample formula:

$$\Phi \wedge \neg P$$

A satisfying model violates the property; unsatisfiability establishes that Φ entails P . Because inconsistent assumptions entail any property vacuously, satisfiability of Φ must also be checked. When temporal states are encoded over finitely many indexed steps, the conclusion applies to that bound and those assumptions. Its scope is determined by the facts and relationships represented in the specification. [2]

Consider an elementary arithmetic example: x is an integer and Φ is ‘ $0 \leq x \leq 5$.’ The assignment $x = 3$ satisfies Φ . Here, a model means an interpretation of logical variables and symbols that satisfies the formula, not a trained neural network. If P is ‘ $x \leq 10$,’ the counterexample query asks whether ‘ $0 \leq x \leq 5$ and $x > 10$ ’ can hold. No integer satisfies it, so the query is UNSAT and $\Phi \models P$ holds.

If instead P is ‘ $x \leq 2$,’ assigning $x = 3$ satisfies Φ but violates P . The counterexample query is SAT, and this assignment refutes the claim that P always holds. SAT for a counterexample query does not mean the original specification is inconsistent: it identifies a case permitted by the specification that violates the property. SAT and UNSAT must therefore be interpreted with respect to the particular query.

3. Finite Temporal Bounds and Assumptions

Temporal change can be represented by indexed states and relations between adjacent states. For example, s_0 , s_1 , and s_2 denote three decision states with two intervening transitions. Unless a time interval is separately defined, these indices do not denote physical seconds. A bounded verification problem combines initial conditions, transition relations, and properties within the selected range.

This distinction matters when interpreting a result. The absence of a counterexample over three states does not establish a property at every future time. Likewise, when observed facts are supplied as assumptions, logical verification addresses what follows

from those assumptions rather than independently establishing their observational accuracy. Specification consistency, property satisfaction, and the correctness of input facts are distinct questions.

C. Controlled Natural Language

Controlled natural language (CNL) deliberately restricts the vocabulary, grammar, or interpretation of a natural language. It supports precision and consistency while retaining natural-language readability. CNLs intended to improve document comprehension differ in purpose and rigor from those designed to correspond to formal representations. [4]

For formally oriented CNL, interpretation rules for conditions, negation scope, and logical relationships are important. Precision depends on explicit definitions of permitted expressions and their interpretation, rather than fluency alone. Describing a CNL therefore requires attention to its particular grammatical and semantic rules. [4]

As a simple illustration, let p mean “the device is locked” and q mean “the device operates.” The sentence “If the device is locked, it does not operate” can be assigned the interpretation $p \rightarrow \neg q$. It restricts operation while locked, but does not require operation whenever the device is unlocked. Rewriting it as “If the device is unlocked, it operates” instead gives $\neg p \rightarrow q$, a different condition. Natural phrasing and logical equivalence are separate issues.

In this example, p and q fix the meanings of terms, “if” expresses the conditional relationship, and “not” determines the scope of negation. An explicit correspondence makes the condition and conclusion identifiable. Preserving meaning between language and formulas concerns these lexical, grammatical, and logical relationships, not merely similar words or sentence lengths. This example illustrates a CNL principle without defining a domain-specific constraint language.

D. Low-Rank Adaptation

Supervised fine-tuning adapts a pretrained model using examples of inputs and target outputs. For language outputs, training can encourage generation of target tokens given an input. Full fine-tuning updates all model weights selected for training, whereas parameter-efficient fine-tuning learns a subset of parameters or a small additional parameter set.

Low-Rank Adaptation (LoRA) fine-tunes a model by freezing pretrained weights and learning an update represented by low-rank matrices. For a pretrained weight matrix W_0 , the adapted weight is expressed as follows. [5]

$$W = W_0 + \frac{\alpha}{r}BA$$

For W_0 of size $d \times k$, the trainable matrices A and B have sizes $r \times k$ and $d \times r$, respectively. Here, r is the rank and α controls scaling. A sufficiently small r reduces the trainable parameter count for each adapted matrix from dk to $r(d + k)$. This decomposition adapts a pretrained model to a task without updating the full weight matrix. [5]

Symbol	Meaning	Dimension or role
W_0	Frozen pretrained weight	$d \times k$ matrix
W	Effective adapted weight	$d \times k$ matrix
k, d	Input and output dimensions of the linear map	Columns and rows, respectively
r	Internal dimension of the low-rank factorization	Positive integer, usually smaller than d and k
A	First trainable factor	$r \times k$ matrix
B	Second trainable factor	$d \times r$ matrix
α	Update scaling coefficient	Scalar setting
BA	Low-rank weight update	$d \times k$ matrix with rank at most r

The dimensions explain the multiplication order: B has r columns and A has r rows, so BA is defined and has the same $d \times k$ shape as W_0 . The update can consequently be added to W_0 . The factor α/r scales its magnitude; r also affects the space of representable updates rather than acting only as a scale factor.

For an illustrative calculation with $d = k = 512$ and $r = 8$, the full matrix contains 262,144 values, whereas A and B together contain 8,192 trainable values, or 3.125% as many. This is a one-matrix example, not a whole-model memory reduction or an experimental setting. LoRA specifies how adaptation is performed; the accuracy and constraint compliance of learned actions require separate evaluation.

References

- [1] NVIDIA et al., “Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail,” arXiv:2511.00088, 2025, revised 2026. [Source](#)
- [2] C. Barrett and C. Tinelli, “Satisfiability Modulo Theories,” Handbook of Model Checking, pp. 305–343, 2018. [Source](#)
- [3] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS, pp. 337–340, 2008. [Source](#)
- [4] T. Kuhn, “A Survey and Classification of Controlled Natural Languages,” Computational Linguistics, vol. 40, no. 1, pp. 121–170, 2014. [Source](#)
- [5] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021. [Source](#)